

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –
Системное и прикладное программное обеспечение

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

«Вводный эксперимент:
сравнение способов обхода графа»

Вариант: Read vs Write, Cache, 10M, Seq

Выполнил:
студент группы Р3310
Чжун Цзяцзюнь

Проверил:
преподаватель
Максимов Андрей Николаевич

1. Цель и постановка эксперимента

Исследовать влияние изменения данных на время последовательного обхода графа размером 10 МиБ при включённом системном кешировании. Сравниваются реализации через `read/lseek` и `mmap`. Изменяемый фактор внутри каждой реализации: чтение или чтение с обновлением поля `value`.

Серия	Реализация	Нагрузка	Аргументы режима
A	<code>read/lseek</code>	Чтение	Без флагов
B	<code>read/lseek</code>	Чтение и запись	<code>--write</code>
C	<code>mmap</code>	Чтение	Без флагов
D	<code>mmap</code>	Чтение и запись	<code>--write</code>

Во всех сериях используются Seq, 10 МиБ и Cache; `--no-cache` не передаётся. По обозначениям практики A/C соответствуют (-, -), B/D соответствуют (WR, -).

Проверяемая гипотеза: при фиксированных Seq, 10 МиБ и Cache среднее время обхода с обновлением `value` больше среднего времени обхода без обновления как у `read/lseek`, так и у `mmap`. Ожидаемая причина: обновление добавляет работу по записи; в первой реализации — дополнительные системные вызовы, во второй — изменение отображённых страниц. Отдельная запись гипотезы до запуска в переданных файлах отсутствует; далее она проверяется без утверждения о предварительной регистрации.

2. Паспорт системы

Параметр	Зафиксированное значение
Дата	28.09.2026, UTC+03:00
ОС и среда	Ubuntu 22.04.5 LTS, WSL2 на Windows; x86_64
Ядро	6.6.87.2-microsoft-standard-WSL2
Модель CPU	13th Gen Intel Core i7-13700H
Топология гостя	20 логических CPU, 10 ядер, 2 потока на ядро по <code>lscpu</code> ; это представление WSL
Закрепление	<code>taskset -c 2</code> ; логический CPU 2
Память гостя	7835,2 МиБ RAM; доступно около 7,0 ГиБ; <code>swap</code> 2 ГиБ, использовано 0
Страница / файл	4096 байт / 10 485 760 байт
Файловая система	ext4, <code>/dev/sdd</code> ; файл в домашнем каталоге Ubuntu
Компилятор	GCC 11.4.0; <code>-O2 -Wall</code>
Частота CPU	<code>cpufreq/governor</code> недоступен в WSL; фиксация частоты и Turbo не подтверждена

Источники: снимки `meta_before.txt`, `meta_after.txt`; версия дистрибутива дополнительно проверена после эксперимента через `/etc/os-release`. Модель физического накопителя не установлена: Virtual Disk, 1 ТиБ и `ROTA=1` относятся к виртуальному устройству и не доказывают наличие HDD. Размер графа составляет около 0,13% доступной гостю общей RAM.

3. Ход работы и методика

Ссылка на выполненную работу: персональный PR или коммит не предоставлен. Исходная постановка: репозиторий курса, intro-exr. Серия данных: series-20260928-235347-14706.

3.1. Подготовка и проверка

Граф создан штатным генератором с параметрами `-s 10M --seed 427 --topology sequential --verify`. Заголовок занимает 40 байт, запись вершины 24 байта; число вершин 436 905. Все вершины достижимы, граф ацикличен. Дополнительно проверены последовательные ссылки: вершина i ведёт к $i+1$, у последней вершины нет потомка.

В режиме записи `value` увеличивается на единицу при каждом посещении; связи графа сохраняются. Использован исходный код курса, совпадающий по SHA-256 со снимками до и после серии. Текущие бинарные файлы также совпадают с зафиксированными хешами.

3.2. Условия одного запуска

Перед каждым запуском `sr` восстанавливает рабочий граф из исходной копии, `sync` завершает предшествующие записи, `cat` читает файл для прогрева страничного кеша. Подготовка выполняется вне таймера. `sync` действует на файловые системы в целом; это не операция отключения кеша.

Каждый процесс выполняет 20 полных обходов. Исходное чистое состояние восстанавливается перед процессом, но не между его 20 обходами: при записи последующие обходы могут работать с уже изменёнными страницами. Этот протокол одинаков для B и D.

`time.monotonic_ns()` измеряет интервал вокруг запуска и завершения цепочки `/usr/bin/time → taskset → программа`. В интервал входят запуск служебных процессов, открытие/закрытие или отображение файла и вывод в журнал. Запуск самого интерпретатора Python происходит до начала таймера. `user/sys`, переключения контекста и ошибки страниц получены через GNU `time`.

3.3. План серии

Для каждой конфигурации выполнены 2 прогревочных запуска, затем $N=30$ измеряемых запусков. Число прогревов задано в сценарии до данной серии; эти запуски отделены от CSV измерений, чтобы начальная подготовка программы и состояния кеша не входила в оцениваемое среднее. Данные не доказывают, что именно два прогрева являются минимально достаточным числом. Независимой единицей анализа принят запуск процесса, а не отдельный обход: 20 обходов не увеличивают статистическое N до 600.

Порядок чередования: A-B-C-D, B-C-D-A, C-D-A-B, D-A-B-C, затем повтор. За 30 раундов каждая группа занимает каждую позицию 7 или 8 раз. Предварительная пилотная серия и расчёт N до сбора данных не представлены. $N=30$ является заданным параметром, его достаточность оценивается после измерений.

Все 120 строк имеют код завершения 0 и `sr_check=ok`. Проверены 120 журналов по 20 успешных обходов, а также 8 прогревочных журналов. Итого в основной серии 2400 обходов. Прогрев не включён в CSV.

4. Статистические результаты

Пусть T_i — записанное время запуска в секундах. Анализируется $x_i = 1000 \frac{T_i}{20}$, то есть нормированное время в миллисекундах на обход. Это среднее время запуска на один обход, а не отдельное инструментальное измерение каждой итерации.

$$|x| = \frac{\sum_{i=1}^N x_i}{N}, \quad s = \sqrt{\frac{\sum_{i=1}^N (x_i - |x|)^2}{N-1}}$$

$$\text{ДИ}_{95\%} = |x| \pm t_{0.975, N-1} \frac{s}{\sqrt{N}}$$

При $N=30$ используется $t=2,04523$ (29 степеней свободы). Интервал характеризует неопределённость среднего при предположении достаточной независимости запусков, но не учитывает систематическую ошибку часов или инструмента.

Серия	N	Среднее, мс/обход	s, мс/обход	95% ДИ, мс/обход
A	30	97,7952	0,6290	97,5603; 98,0300
B	30	240,5746	1,1429	240,1478; 241,0013
C	30	1,5607	0,0990	1,5237; 1,5976
D	30	3,3886	0,1567	3,3301; 3,4471

Средние времена одного запуска из 20 обходов: A — 1,955903 с; B — 4,811491 с; C — 0,031213 с; D — 0,067772 с.

Четыре серии: среднее и 95% ДИ, $N = 30$

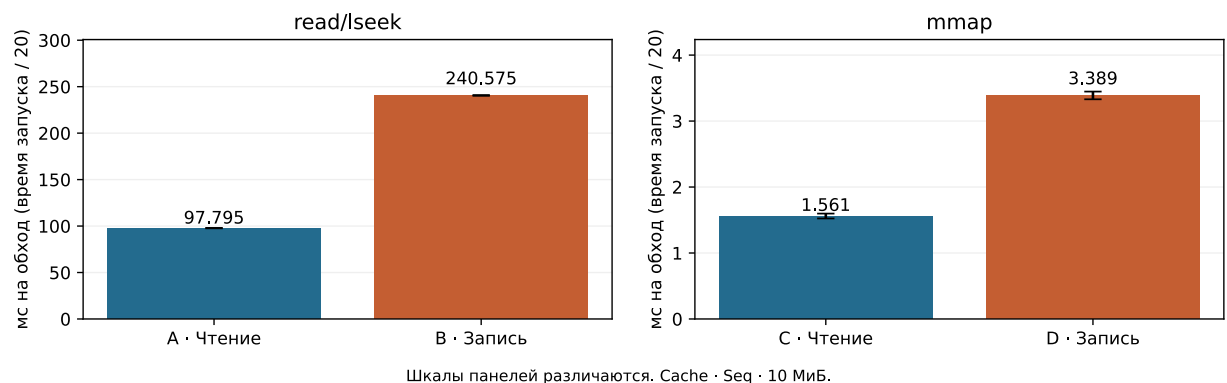


Рис. 1. Средние четырёх серий и 95% доверительные интервалы. Шкалы панелей различаются; внутри каждой панели шкала линейная и начинается с нуля.

Для A/B и C/D интервалы средних не пересекаются. В переданных данных запись занимает больше времени в обеих реализациях. Это вывод о зарегистрированных измерениях; ограничение проверки часов изложено в разделе 6.

Серия	Медиана, мс/обход	IQR, мс/обход	Полуширина ДИ, % среднего
A	97,8651	1,0828	0,2402%
B	240,2837	1,2037	0,1774%
C	1,5183	0,1084	2,3691%
D	3,3487	0,1527	1,7269%

5. Сравнение и объяснение результатов

5.1. Влияние записи

- **read/lseek:** B/A=2,4600. Запись увеличивает среднее время на 142,7794 мс/обход, или на 146,00% относительно чтения.
- **mmar:** D/C=2,1713. Увеличение составляет 1,8279 мс/обход, или 117,13%.

Нагрузка Write включает чтение структуры графа и изменение значений; это не сравнение чистого чтения с независимым потоком только записей. В реализации read/lseek обновление требует дополнительных обращений к ядру. При mmar изменение происходит через отображённую память и включает обработку доступа к записываемым страницам. Эти объяснения следуют из кода и согласуются со счётчиками; результаты strace приведены в диагностическом разделе.

5.2. Сопоставление реализаций

Отношение средних read/lseek к mmar равно 62,6628 для чтения и 70,9954 для записи. Это характеристики данного графа, сборки и протокола; они не означают универсального ускорения mmar во всех задачах. Число обходов одинаково во всех группах, однако относительный вклад запуска процессов особенно заметен в коротких C/D.

У read/lseek каждая вершина обрабатывается через системные вызовы позиционирования и чтения. mmar позволяет обращаться к отображённым данным без отдельного read для каждой вершины. Ошибка страницы при доступе к отображению и системный вызов read — разные события.

5.3. Ресурсные показатели

В таблице приведены средние **на один процесс, выполняющий 20 обходов**, без деления на число итераций. Дробные счётчики возникают при усреднении целых значений.

Серия	user, c	sys, c	minor faults	major	Добров. CS	Вынужд. CS
A	0,5990	1,1693	138,23	0	1,93	6,27
B	1,2660	3,0973	137,87	0	1,83	14,07
C	0,0127	0,0047	3337,83	0	1,60	0,13
D	0,0203	0,0313	54595,70	0	1,73	0,33

У B системное время выше, чем у A, примерно в 2,65 раза. У mmar число малых ошибок страниц существенно больше, особенно при записи. Это совместимо с повторным созданием отображений и обработкой доступа к страницам. Малые ошибки не означают чтение с диска; нулевое число больших ошибок согласуется с прогревом, но само по себе не доказывает отсутствие любых операций ввода-вывода.

Счётчики CS отражают переключения контекста, а не каждый переход в ядро. Значения user/sys записаны с шагом 0,01 с, поэтому для C/D точность их соотношений ограничена; sys=0.00 не означает полного отсутствия системной работы.

6. Устойчивость и ограничения данных

6.1. Порядок измерений и отклоняющиеся точки

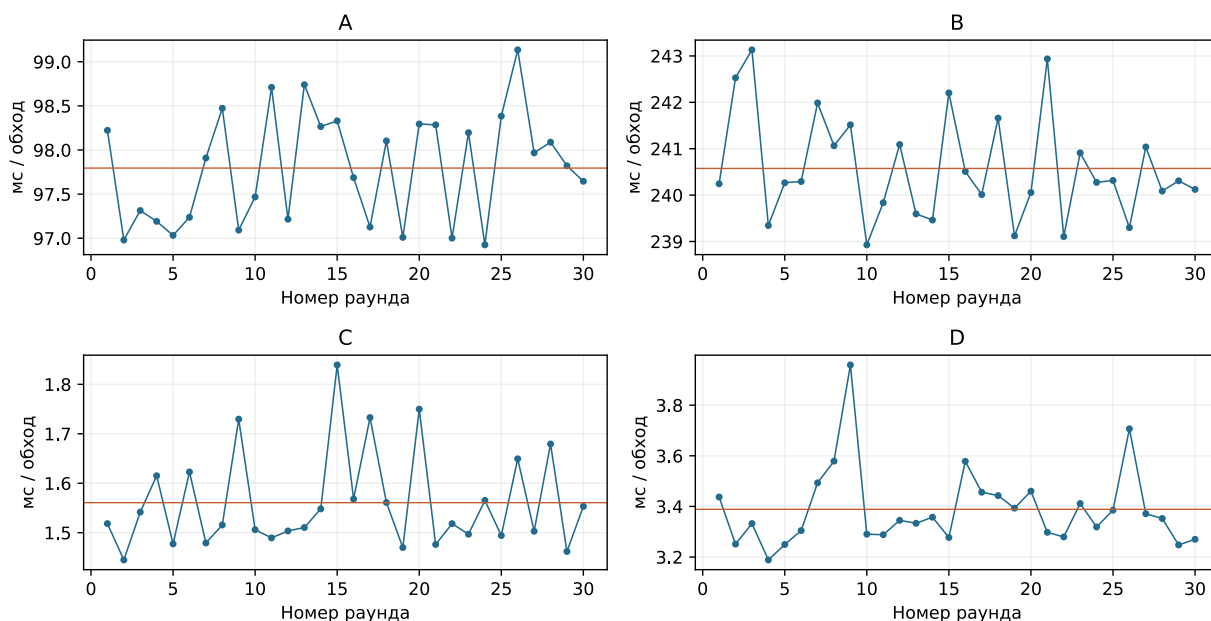


Рис. 2. Нормированное время в порядке раундов. Горизонтальная линия показывает среднее; оси Y независимы.

Среднее второй половины относительно первой изменилось на +0,101% (A), -0,159% (B), +0,591% (C), +0,562% (D). Эти небольшие сдвиги не доказывают независимости наблюдений.

По диагностическому правилу $1,5 \times \text{IQR}$ отмечены раунды 3 и 21 в B, раунд 15 в C, раунды 9 и 26 в D; в A отмеченных точек нет. **Все 120 точек сохранены.** Правило использовано только для описания, не для удаления значений.

6.2. Проверка двух шкал времени

Для каждого из 120 основных запусков журнал содержит интервалы CLOCK_REALTIME и CLOCK_MONOTONIC. В 37 запусках их разность по модулю превышает 1 мс; максимальное расхождение равно 8,593230 с. Календарные метки CSV пять раз идут назад. Сумма монотонных интервалов составляет 205,991385 с, а календарные снимки 23:54:08 и 23:57:12 дают 184 с.

В диагностике расхождение более 1 мс имеется в 5 из 24 запусков, максимум 6,089550 с. Таким образом, несогласованность шкал подтверждена непосредственно. Её причина не установлена; по этим данным нельзя независимо подтвердить абсолютную точность самого монотонного таймера. Автор сообщил о просмотре данных через sat, без редактирования.

Расчёты используют только сохранённое wall монотонного таймера; календарные метки не используются для вычитания времени или сортировки. Данные не корректировались. Узкие t-интервалы характеризуют случайную вариацию записанного wall, но не устраняют возможную систематическую ошибку часов.

7. Диагностика и влияние инструментов

Источник: diag-short-20260928-235755-16397, STAGE=all. Проверены 24 записи с valid=1 и кодом завершения 0: 12 инструментальных запусков и 12 запусков с/без нагрузки. В журналах подтверждено ожидаемое число успешных обходов; два дополнительных прогрева не входят в эти 24 записи. Результаты старых серий не объединены с текущими.

7.1. Системные вызовы

Один обход на процесс; каждая конфигурация прослежена один раз. taskset выполняется до strace; счётчики включают загрузчик и сообщения программы. Поэтому их нельзя целиком приписать операциям с вершинами.

Вызов	A	B	C	D
read	436 907	436 907	1	1
lseek	436 906	873 811	0	0
write	2	436 907	2	2
mmap	8	8	9	9
munmap	1	1	2	2
madvise	0	0	0	0
Всего	873 850	1 747 660	40	40

В B на 436 905 больше lseek и на 436 905 больше write, чем в A: это соответствует одному обновлению каждой вершины. В C/D нет read/write на каждую вершину; запись в MAP_SHARED изменяет отображённые страницы. Два write в C/D относятся к сообщениям, а не к отдельной записи каждой вершины. Вызов madvise в обычном Cache-режиме отсутствует: в исходном коде он находится в ветке no-cache.

7.2. Накладные расходы

Во всех строках ниже один обход; wall измерен внешним монотонным таймером. «Без инструмента» всё равно включает общий запуск timeout и taskset. На каждый инструмент и конфигурацию имеется только одна точка; средние по повторной выборке и ДИ здесь не вычисляются.

Серия	Без инструмента, мс	time -v, мс	strace, мс	strace / без инструмента
A	103,747	99,968	4353,812	41,966
B	250,188	247,399	8868,146	35,446
C	4,644	4,181	6,867	1,479
D	6,586	7,055	7,519	1,142

Для A/B замедление под strace намного сильнее, чем для C/D, что согласуется с числом перехватываемых системных вызовов. Иногда time -v оказался быстрее единичного запуска без него; это не означает ускорения программ инструментом. Порядок фиксирован и N=1, поэтому разность может отражать вариацию запуска и состояния системы. Основная серия выполнялась без strace и perf.

8. Ресурсная диагностика и фоновая нагрузка

8.1. GNU time и недоступный perf

Следующие значения time -v относятся к **одному обходу**, в отличие от 20 обходов в основной серии. Их нельзя напрямую сопоставлять без учёта разного объёма работы и запуска процесса.

Серия	user/sys, с	Max RSS, КиБ	Малые ошибки	Добров. CS	Вынужд. CS
A	0,04 / 0,04	1440	69	1	0
B	0,06 / 0,16	1440	71	1	0
C	0,00 / 0,00	11680	232	1	0
D	0,00 / 0,00	11680	2790	1	0

Большие ошибки страниц во всех четырёх запусках равны нулю. Округление user/sys до 0,00 у mmap не доказывает отсутствия процессорной работы. Max RSS отражает резидентную память процесса, а не всю память, занятую файловым кешем системы.

В диагностике 28.09.2026 проверка perf завершилась сообщением «perf not found for kernel 6.6.87.2-microsoft». Обёртка не нашла ожидаемый инструмент; это не доказательство запрета всех событий в WSL. В той серии значения perf не получены и не заменены нулями. Дополнительная проверка 07.10.2026 обнаружила работоспособный отдельный бинарный файл: программные события получены, аппаратное событие cache-misses не поддерживается. Новые данные приведены в разделе 10. Закрепление CPU само по себе не является измерением числа миграций.

8.2. Влияние конкуренции за CPU

Для каждой реализации выполнены три пары Read без нагрузки / с нагрузкой, по 20 обходов. Нагрузка — один Bash busy loop на том же логическом CPU 2. Она запускается после cp/sync/cat и завершается после измерения. Порядок quiet/stress чередуется по повторениям. Это заданный опыт конкуренции за один CPU, не доказанный «наихудший случай» для всех систем.

Реализация	Условие	Среднее wall, с	s, с	Средние вынужд. CS
read/lseek	Без нагрузки	1,956309	0,013403	7,67
read/lseek	CPU-нагрузка	4,281070	0,039363	533,67
mmap	Без нагрузки	0,036017	0,005826	0,67
mmap	CPU-нагрузка	0,068207	0,007510	7,67

Среднее выросло в 2,1883 раза у read/lseek и в 1,8937 раза у mmap. Число вынужденных переключений также выросло. Это согласуется с разделением процессорного времени между конкурирующими задачами. По трём парам нельзя точно оценить эффект произвольной фоновой программы или переносить коэффициент на любую машину.

95% t-интервалы среднего wall: read/lseek — 1,9230–1,9896 с без нагрузки и 4,1833–4,3789 с с нагрузкой; mmap — 0,021545–0,050490 с и 0,049551–0,086864 с. У mmap интервалы слегка пересекаются; наблюдаемый рост среднего не следует выдавать за отдельное доказательство статистической значимости. Само пересечение также не является универсальным тестом равенства.

9. Достаточность измерений и выводы

Для ретроспективной проверки принят ориентир: полуширина 95% ДИ не более 5% среднего. Это критерий анализа после сбора, а не заявленная заранее цель. Во всех четырёх сериях наблюдаемая полуширина ниже 5%; различие Read/Write значительно больше случайного разброса среднего.

При неизменном s увеличение N с 30 до 60 уменьшит полуширину приблизительно в 0,69 раза с учётом t -квантиля. Формальный расчёт $t \frac{s}{\sqrt{N}} \leq 0.05|x|$ с текущими оценками s даёт $N \approx 3, 2, 9, 6$ для A–D. Эти малые числа не являются рекомендацией проводить эксперимент на 2–3 точках: дисперсия оценена по 30 измерениям, а устойчивость и систематические ошибки формула не проверяет.

Более важная следующая проверка — согласованность часов и накладные расходы коротких запусков, а не механическое увеличение N . У S средний процесс длится лишь 31,2 мс; разрешение монотонного таймера устраняет округление wall до нуля, но не устраняет стоимость запуска и мониторинга.

9.1. Ограничения выполнения задания

- Снимки показывают рост load average 0,22/0,05/0,04 $\rightarrow$ 0,98/0,54/0,24; это может включать нагрузку самой серии. Снимки гостя не гарантируют отсутствия фоновой нагрузки Windows.
- Governor недоступен; постоянная частота CPU, температура и состояние Turbo не подтверждены. Закрепление гостевого логического CPU не гарантирует неизменного физического ядра Windows.
- Основная серия, четыре инструментальных конфигурации и фоновый опыт для обеих реализаций выполнены. В исходной диагностике на инструмент $N=1$, на условие нагрузки $N=3$. Повторная диагностика с $N=3$ на инструмент приведена отдельно в разделе 10; аппаратное событие cache-misses осталось недоступным.

9.2. Общий вывод

1. В полученных записях обновление значений увеличивает время обхода в 2,4600 раза для read/lseek и в 2,1713 раза для mmap; проверяемое предположение согласуется с данными.
2. mmap имеет существенно меньшие зарегистрированные времена для этого последовательного графа. Анализ исходного кода связывает это с доступом через отображение вместо системных вызовов на каждой вершине; трассировка подтверждает различие числа системных вызовов, но не выделяет точный временной вклад каждого механизма.
3. Разброс средних относительно различия режимов невелик. Однако узкий доверительный интервал не устраняет несогласованность часов и возможную систематическую ошибку.
4. Cache-режим не требует обязательного fsync/msync после каждого измеряемого обновления. Результат характеризует выполнение программы с кэшированием, а не полную задержку долговечной записи на физический носитель.
5. Диагностика подтверждает, что strace существенно меняет наблюдаемое время прежде всего у реализации с большим числом вызовов. Фоновая конкуренция увеличила среднее время обеих реализаций; поэтому результаты шумных запусков не смешивались с основной серией.
6. Основные этапы сбора и анализа выполнены. Ограничения стенда и отсутствие предварительной оценки N отражены явно. Неполная поддержка событий perf и окончательная трактовка расхождения часов должны учитываться при защите; вымышленные значения в отчёт не включены.

Материалы: обе исходные серии и дополнительная диагностика находятся в report-data/final-review/; таблицы основной серии — в report-assets/final/. Исходные сценарии сохранены внутри каталогов результатов. Таблица средних, s и ДИ воспроизводится скриптом report-assets/analyze_results.awk для $N=30$, ITER=20. Ранее выполненные серии и отдельный запуск STAGE=noise не объединялись с выбранными данными.

10. Дополнительная проверка инструментов, 07.10.2026

Источник: diag-short-20261007-162239-485. Цель — дополнить исходную диагностику повторными измерениями и проверить доступность perf. Получены 48 корректных запусков: 4 конфигурации × 4 инструмента × 3 повтора. Каждый измеряемый процесс выполняет один обход; два предварительных прогрева по 20 обходов исключены. Инструменты внутри каждой конфигурации чередуются с циклическим сдвигом порядка по повторениям. Протокол восстановления файла и прогрева page cache сохраняется.

Метрика wall включает запуск timeout, taskset и выбранного инструмента. Ниже показаны средние и 95% t-интервалы в миллисекундах на **один запуск с одним обходом**. При N=3 квантиль равен 4,30265; расчёт выполнен в AWK внутри сценария и независимо проверен по CSV. Это диагностические измерения другой даты: они не добавлялись к основным выборкам N=30.

Серия	Инструмент	N	Среднее, мс	95% ДИ, мс
A	Без инструмента	3	95,819	85,589; 106,050
A	time -v	3	97,763	92,151; 103,376
A	strace	3	4269,392	4001,118; 4537,665
A	perf	3	126,170	97,777; 154,564
B	Без инструмента	3	229,200	227,278; 231,122
B	time -v	3	236,564	219,612; 253,516
B	strace	3	8466,539	7938,620; 8994,458
B	perf	3	263,034	242,149; 283,918
C	Без инструмента	3	3,561	2,974; 4,148
C	time -v	3	4,246	2,460; 6,032
C	strace	3	5,370	3,282; 7,457
C	perf	3	26,668	-8,814; 62,149
D	Без инструмента	3	5,708	5,463; 5,954
D	time -v	3	6,222	5,022; 7,421
D	strace	3	7,267	6,977; 7,556
D	perf	3	17,191	-3,547; 37,929

Отношения средних strace / без инструмента: A — 44,5567; B — 36,9396; C — 1,5080; D — 1,2730. У A/B интервалы strace и минимального измерения сильно разделены. У C они пересекаются: точечный рост не доказывает точно установленного эффекта. Интервалы perf для C/D очень широки; отрицательные нижние границы являются следствием симметричной t-аппроксимации при малой выборке, а не отрицательным физическим временем. Границы не обрезались. Три точки не обеспечивают точной оценки малых накладных расходов.

Вся дополнительная проверка заняла около 61 секунды по календарным снимкам. Полный журнал и точные стандартные отклонения сохранены в measurements.csv и summary.csv. Резкое замедление read/lseek под strace согласуется с перехватом сотен тысяч системных вызовов, поэтому strace применяется для диагностики отдельно от основной серии.

10.1. Доступные события perf и состояние стенда

Использован /usr/lib/linux-tools-5.15.0-194/perf, версия 5.15.209. Хотя его версия отличается от ядра 6.6.87.2, фактическая проверка программных событий завершилась успешно. Исходная обёртка /usr/bin/perf продолжает выдавать ошибку поиска версии для WSL-ядра. В журналах события имеют суффикс :u, отражающий пользовательский фильтр при текущих правах; нельзя подменять их показателями всей системы или всех режимов CPU.

Серия	context-switches:u	cpu-migrations:u	page-faults:u	cache-misses:u
A	0 / 0 / 0	0 / 0 / 0	55 / 56 / 57	Не поддерж.
B	0 / 0 / 0	0 / 0 / 0	56 / 57 / 57	Не поддерж.
C	0 / 0 / 0	0 / 0 / 0	215 / 217 / 217	Не поддерж.
D	0 / 0 / 0	0 / 0 / 0	2777 / 2776 / 2775	Не поддерж.

В каждой ячейке приведены значения трёх повторов одного обхода. Нулевые значения — реально выданные программные счётчики с указанным фильтром, а не доказательство отсутствия любых переключений на стенде. Для cache-misses инструмент вывел <not supported>; успешный код завершения perf не делает это событие доступным. Страничные ошибки не тождественны промахам кеша CPU. Для анализа переключений также сохранены показатели GNU time из основной серии.

Проверка Windows показала CPU i7-13700H с 14 физическими ядрами и 20 логическими процессорами; WSL представляет гостевую топологию как 10 ядер × 2 потока. Два накопителя хоста определены командой Get-PhysicalDisk как SSD/NVMe: UMIS RPEYJ1T24MKN2QWY и Fanxiang S790E 1TB. Соответствие конкретного файла виртуального диска одному из них не установлено. Эти сведения получены 07.10.2026; их нельзя выдавать за снимок условий серии 28.09.2026.

Граф и программы находятся в /home/ivanstylish/course-code/lab/intro-exp, на ext4, а не на примонтированном Windows-разделе. При начальной проверке доступно около 6,9 ГиБ памяти гостя, swapon не используется; привязка к логическому CPU 2 работоспособна. Governor в госте отсутствует. Windows использует пользовательский план питания; сведения батареи указывают подключённое питание и заряд 100%. Это не подтверждает фиксацию частоты или отсутствие Turbo/троттлинга. Краткая выборка Windows показывает небольшую активность фоновых приложений, а не абсолютную изоляцию.

Проверка времени: в новых 48 запусках расхождение realtime и monotonic больше 1 мс отмечено 5 раз; максимум — 5,406445 с. Поэтому ограничение раздела 6 остаётся актуальным. В отдельном контрольном запуске strace внешний таймер Windows показал 9,1498 с, внутренний monotonic — 8,9873 с, realtime — 8,9863 с. Внешний интервал дополнительно включает вызов WSL и подготовку файла, поэтому это лишь проверка масштаба времени одного запуска. Она не подтверждает точность всех предыдущих измерений и не устанавливает причину скачков.

Вывод дополнительной проверки: повторная оценка инструментального воздействия выполнена; программные события perf получены. Поддержка аппаратного cache-misses и согласованность часов остаются ограничениями стенда. Исходные временные ряды, выбор N=30 и отсутствие документированного предварительного pilot не изменялись и не представлялись как заново спланированный опыт.